LAB 07 Tic-Tac-Toe

October 26, 2026

[ ]:
```
# We first import from a useful packages that will be required in this lab
import numpy as np
from random import seed
from random import random
```

1 A longer coding exercise - pulling things together

This lab combines everything we've done so far. If you didn't understand any of the previous exercises and labs, you may want to ask questions about them before beginning this one!

1.0.1 2D Arrays - lists of lists

You have dealt with numerical arrays from the Numpy library, although normal Python lists can often be used as well (although without the 'deal-with-each-element-in-one-go' power of Numpy). Just as a one-dimensional array is an ordered sequence of values, a two-dimensional array is an ordered sequence of one-dimensional arrays. A list of lists

`Python myArray = [['1','2','3'],['4','5','6'],['7','8','9']]`

An example of the use of 2D lists of lists can be seen below

[ ]:
```
def printArray(values):           # note, has to work out dimensions itself !
  for i in range( len(values) ):
        for j in range( len(values[0]) ):
            print(" {:2} ".format(values[i][j]), end='')
        print(" ")                # just take a newline


myArray = [[0,0,0],[0,0,0],[0,0,0],[0,0,0],[0,0,0]]   # 5 rows, 3 columns

# numerical array in this case: fill it with some different numbers
for i in range(5):
   for j in range(3):
        myArray[i][j] = 3*i + j;

# print, as defined above
printArray(myArray)
```

1.0.2 Preliminaries

Write a program which creates a "game board". Your game board should be 3x3. Each square in the board can either be empty (print a dot "."), or have an "X":

```
. X .
X . .
. . .
```

You must use a list of lists to represent the game board. (you can't have variables like `squareUpperLeft`, `squareBottomMiddle` …)

- The board begins complete clear (all blank squares '.')
- The player may select any square by entering a number from the computer keypad. (your program should read it as an int, and you may assume that it is between 1 and 9 inclusive)
- Before each player takes a move, print out the current game board. You can remove board from the screen by doing `clear_output()` from the `IPython.display` library.
- If the player tries to select a square that is currently occupied, print a warning message and ask them to choose again.
- Continue playing the game until all squares are filled.

It is best to create formulae to deal with "computer keypad <=> row and column". Hint: try formulae to deal with "number <=> row and column" for these three cases:

```
(a)         (b)           (c)
easy        mobile        computer keypad
0 1 2       1 2 3         7 8 9             < row 0
3 4 5       4 5 6         4 5 6             < row 1
6 7 8       7 8 9         1 2 3             < row 2

^ ^ ^       ^ ^ ^         ^ ^ ^
0 1 2       0 1 2         0 1 2
column      column        column
```

Your assignment needs to work with the computer keypad, but it is strongly recommended that you learn how to solve (a) and (b) before trying to do (c). Solve case (a) first. You will find the division / and modulus % (or "remainder") operators useful. After solving (a), it should be possible to figure out how to do (b) and then (c), by tweaks to your formulae

[ ]:
```
from random import seed
from random import random
from IPython.display import clear_output

# tic-tac-toe preliminaries
```

2 Exercise : tic-tac-toe (5 marks)

Write a tic-tac-toe game (also known as *noughts and crosses*, *tick tack toe*, *X's and O's*). If you're not familiar with the game, see Wikipedia's page on tic-tac-toe, or ask a lab instructor.

- Use your game board from above.

- You must use a two-dimensional array (a list of lists) to represent the game board in memory.
- Modify your "print the game board" function to print out .XO as required.
- Your game should be human-vs-computer, and human moves first.
- Computer moves randomly. Note that it must select an unoccupied square; you cannot simply pick a number from 1-9 at random without checking!
- Before each player takes a move, print out the current game board. You can remove board from the screen by doing `clear_output()` from the `IPython.display` library.
- After each player takes a move, you must check to see if anybody won the game.
  - (hint: write a function that checks if player X won the game, then call this function twice, for player 1 and player 2)
  - (another hint: there are 8 possible winning combinations; a player wins if one of 8 combinations of 3 square all have his mark. You can either write out all 8 combinations in full, or use `for` loops – but make sure that you test the three vertical wins, three horizontal wins, and two diagonal wins.)
  - (final hint: begin work on this point by changing the rules of the game. Pretend that you can only win by going horizontally or diagonally across the middle. Solve this problem first; once you have it working for the "fake rules", just add in the other 6 winning combinations.)

(optional: instead of having the computer move randomly, try to make it play intelligently.) (optional: instead of writing procedural code, try and make the tic-tac-toe board an Object, with methods for drawing, checking the state of the board, and accepting user inputs)

[ ]:
```
from random import seed
from random import random
from IPython.display import clear_output

# tic-tac-toe game
```